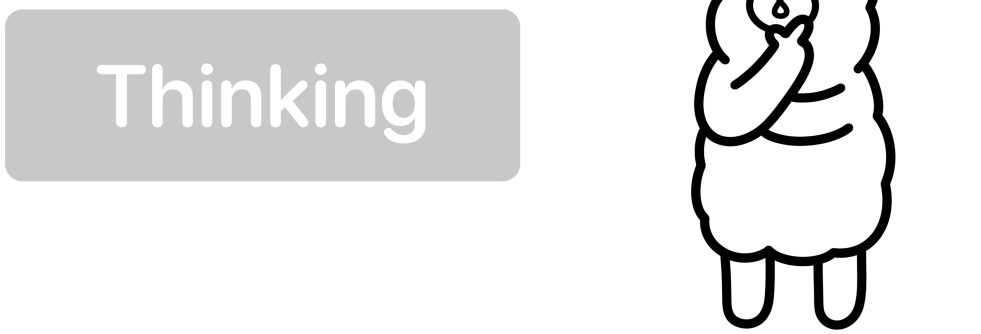
Thinking

May 30, 2025



Ollama now has the ability to enable or disable thinking. This gives users the flexibility to choose the model's thinking behavior for different applications and use cases.

When thinking is enabled, the output will separate the model's thinking from the model's output. When thinking is disabled, the model will not think and directly output the content.

Models that support thinking:

- DeepSeek R1
- Qwen 3
- more will be added under thinking models.

Thinking in action

Enable thinking in DeepSeek R1

In the CLI, thinking is enabled by default.

This can be useful in getting the model to think through different viewpoints to arrive at more accurate answer.

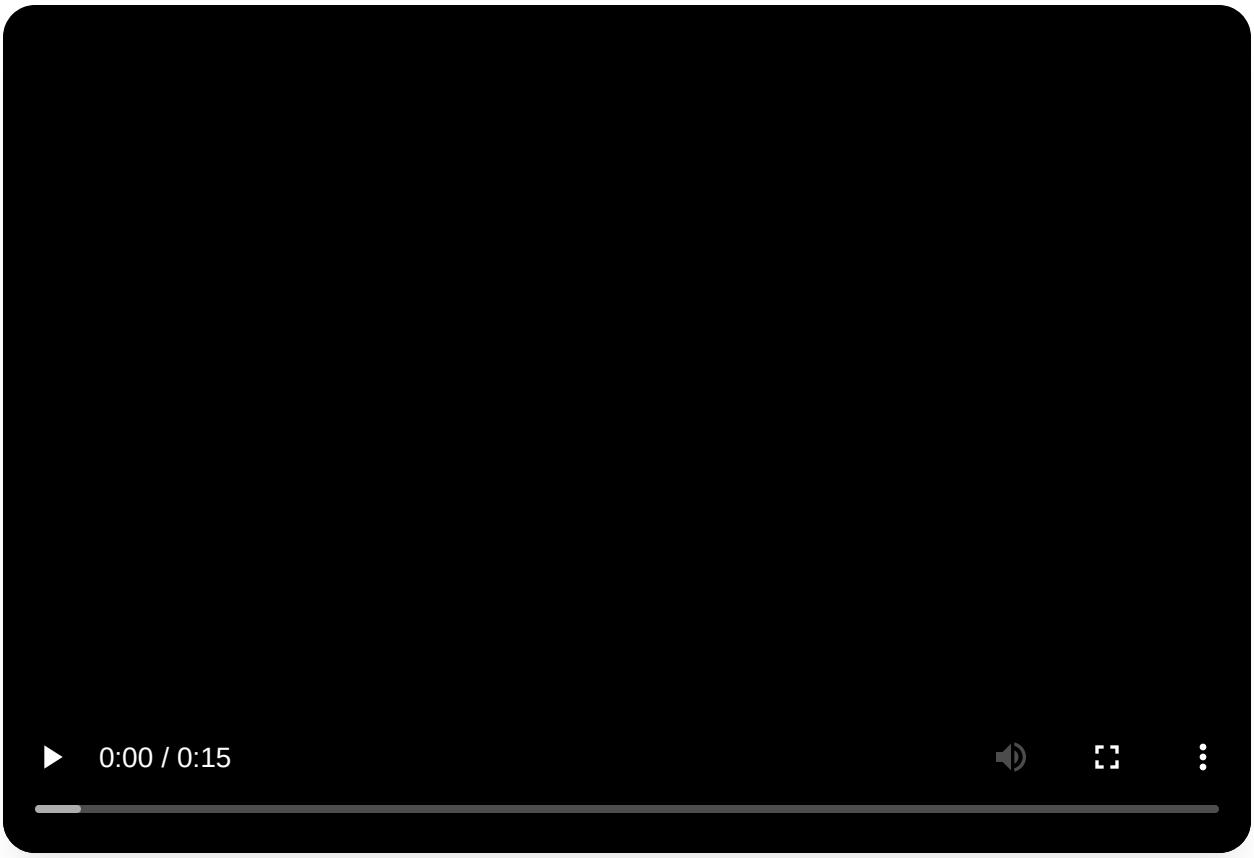


The model shown is the 8 billion parameter DeepSeek-R1-0528 Qwen 3 distilled model. This video is not sped up.

Disable thinking in DeepSeek R1

In the CLI, thinking is disabled using `/set nothink` followed by the prompt.

This is useful in getting answers fast out of the model.



The model shown is the 8 billion parameter DeepSeek-R1-0528 Qwen 3 distilled model. This video is not sped up.

Get started

Download the latest version of Ollama.

CLI

From the Ollama CLI, thinking can be enabled or disabled:

Enable thinking

```
--think
```

Disable thinking

```
--think=false
```

Interactive sessions

When chatting inside an interactive session, thinking can be enabled or disabled:

Enable thinking

```
/set think
```

Disable thinking

```
/set nothink
```

Scripting

For scripting, a `--hidethinking` command is available. This helps users who want to use thinking models but simply want to see the answer.

Example:

```
ollama run deepseek-r1:8b --hidethinking "is 9.9 bigger or 9.11?"
```

API

Both of Ollama's generate API (`/api/generate`) and chat API (`/api/chat`) have been updated to support thinking.

There is a new `think` parameter that can be set to `true` or `false` for enabling a model's thinking process. When the `think` parameter is set to true, the output will separate the model's thinking from the model's output. This can help users craft new application experiences like animating the thinking process via a graphical interface, or for NPCs in games to have a thinking bubble before the output. When the `think` parameter is set to false, the model will not think and directly output the content.

Example using Ollama's chat API with thinking enabled

```
curl http://localhost:11434/api/chat -d '{
  "model": "deepseek-r1",
  "messages": [
    {
      "role": "user",
      "content": "how many r in the word strawberry?"
    }
  ],
  "think": true,
  "stream": false
}'
```

Output


```
{"model":"deepseek-r1",
"created_at":"2025-05-29T09:35:56.836222Z",
"message":
    {"role": "assistant",
    "content": "The word \"strawberry\" contains **three** instances of th
    "thinking": "First, the question is: \"how many r in the word  strawbe
    "done_reason":"stop",
    "done":true,
    "total_duration":47975065417,
    "load_duration":29758167,
    "prompt_eval_count":10,
    "prompt_eval_duration":174191542,
    "eval_count":2514,
    "eval_duration":47770692833
    }
}
```


Output is truncated for brevity.

Python library

Please update to the latest Ollama Python library.

```
pip install ollama
```

Example of enabling thinking

```python
from ollama import chat

messages = [
  {
    'role': 'user',
    'content': 'What is 10 + 23?',
  },
]

response = chat('deepseek-r1', messages=messages, think=True)

print('Thinking:\n========\n\n' + response.message.thinking)
print('\nResponse:\n========\n\n' + response.message.content)
```

Please visit the Ollama Python library for more information about its usage. More examples are available.

JavaScript library

Please update to the latest Ollama JavaScript library.

```
npm i ollama
```

Example of enabling thinking

```
import ollama from 'ollama'

async function main() {
  const response = await ollama.chat({
    model: 'deepseek-r1',
    messages: [
      {
        role: 'user',
        content: 'What is 10 + 23',
      },
    ],
    stream: false,
    think: true,
  })

  console.log('Thinking:\n========\n\n' + response.message.thinking)
  console.log('\nResponse:\n========\n\n' + response.message.content + '\r
}

main()
```

Example of streaming responses with thinking

```
import ollama from 'ollama'

async function main() {
  const response = await ollama.chat({
    model: 'deepseek-r1',
    messages: [
      {
        role: 'user',
        content: 'What is 10 + 23',
      },
    ],
    stream: true,
    think: true,
  })

  let startedThinking = false
  let finishedThinking = false

  for await (const chunk of response) {
    if (chunk.message.thinking && !startedThinking) {
      startedThinking = true
      process.stdout.write('Thinking:\n========\n\n')
    } else if (chunk.message.content && startedThinking && !finishedThinki
      finishedThinking = true
      process.stdout.write('\n\nResponse:\n========\n\n')
    }

    if (chunk.message.thinking) {
      process.stdout.write(chunk.message.thinking)
    } else if (chunk.message.content) {
      process.stdout.write(chunk.message.content)
    }
  }
}

main()
```

Please visit the Ollama JavaScript library for more information about its usage. More examples are available.

Reference

- GitHub

© 2026 Ollama